哈爾濱工程大學

第 13 讲 / 全 17 讲

C 语言排序与查找详解

把一堆乱数整理有序，再飞快地找到它

冒泡排序

相邻两个比一比，大的往后沉

选择排序

每趟挑最小的，放到最前面

顺序查找

从头到尾扫一遍，找到就喊停

二分查找

有序时每次砍掉一半，百次变二十次

主讲教师：郎大鹏

本节目标

哈爾濱工程大學

1. 说清 " 排序 " 是什么：把无序的一串数变成有序
2. 掌握冒泡排序：相邻比较、大的沉底、双重 for
3. 掌握选择排序：每趟选最小的放到前面
4. 会写顺序查找，理解 " 找不到返回 -1" 的约定
5. 掌握二分查找的前提与 low / mid / high 写法
6. 能独立写出 " 先排序、再查找 " 的综合小程序

01

冒泡排序

相邻比较、大的沉底 · 最直观的一种排序

排序是什么　沉底思想　双重 for

哈爾濱工程大学

排序是什么

排序：把一串杂乱的数据，按照某个规则重新排好。最常见的规则是「从小到大」，叫升序。

排好序之后，找最大最小、找某个数、去掉重复、排名次，全都变简单了。

排序前

5 3 8 1 9

乱序：看不出谁最大

找 1 要从头翻到尾

升序排好

1 3 5 8 9

最小在最左，最大在最右

一眼就能看出极值

降序排好

9 8 5 3 1

从大到小，名次表常用

第 1 名排在最前面

排序规则

关键字

按什么比大小，

就叫按什么排序

冒泡的思想

相邻比较 · 大的沉底

哈爾濱工程大學

第一步：相邻两个比大小

5	3	8	1	9

a[0]=5 > a[1]=3 → 前面大，交换

3	5	8	1	9

再比 a[1] 与 a[2]，再比 a[2] 与 a[3]…… 一直比到末尾

一趟走完：最大的沉到最后

3	5	1	8	9

9 最大，一路被换到末尾，位置就此确定

名字的由来

小的数像气泡，一趟一趟慢慢往上浮；大的数像石头，一路往下沉。

只跟邻居比

每一对只看 a[j] 和 a[j+1] 这两个相邻的，前面比后面大就交换。

一趟定一个

每趟结束，当前还没排好的数里最大的那个，一定到达最终位置。

逐趟图解：5 个数怎么排好

初始	5	3	8	1	9	乱序的一串数，目标是排成从小到大
第 1 趟	3	5	1	8	**9**	比较 4 次，最大的 9 一路沉到最后
第 2 趟	3	1	5	**8**	**9**	比较 3 次，8 归位；末尾的 9 不再参与
第 3 趟	1	3	**5**	**8**	**9**	比较 2 次，5 归位，整串其实已经有序
第 4 趟	**1**	**3**	**5**	**8**	**9**	比较 1 次，确认有序，排序结束

小结 5 个数排了 4 趟（n - 1 趟）；每趟比较 4、3、2、1 次，依次递减

第一趟完整追踪

j 与 j+1 的每一次比较

初始	5	3	8	1	9	i = 0 ，j 从 0 一直走到 3
j = 0	**3**	**5**	8	1	9	a[0]=5 > a[1]=3 ，前面大 → 交换
j = 1	3	5	8	1	9	a[1]=5 < a[2]=8 ，顺序对 → 不动
j = 2	3	5	**1**	**8**	9	a[2]=8 > a[3]=1 ，前面大 → 交换
j = 3	3	5	1	8	9	a[3]=8 < a[4]=9 ，顺序对 → 不动
本趟结束	3	5	1	8	**9**	共比较 4 次，最大 9 到位；后面只剩 4 个数要排

几趟？每趟几次？

两个循环边界

外层 i 管趟数：n 个数最多排 n - 1 趟。每趟结束就有一个数归位，剩下的最后一个数自然是最小的，不用再排。

内层 j 管比较：第 i 趟时末尾已经有 i 个数归位，所以只需要比较 n - 1 - i 次，一趟比一趟少。

外层跑几趟	第 i 趟比几次	内层循环条件	总比较次数
n - 1	**n-1-i**	**j < n-1-i**	**n(n-1)/2**
i 从 0 走到 n-2 最后一个数自动归位	末尾多一个归位， 就少比一次	写成 j < n 会越界， a[j+1] 读到数组外	4+3+2+1 = 10 次（n=5） n 翻倍，工作量变 4 倍

冒泡排序完整代码

双重 for + swap

哈尔滨工程大学

```c
void swap(int *x, int *y) {
    int t = *x;
    *x = *y;
    *y = t;
}   // 指针版 swap 回来了
void bubble(int a[], int n) {
    int i, j;
    // 外层：共 n-1 趟
    for (i = 0; i < n - 1; i++)
        // 内层：每趟少比一次
        for (j = 0; j < n-1-i; j++)
            if (a[j] > a[j+1])
                swap(&a[j], &a[j+1]);
```

swap 用指针才有效

函数课那个换了没用的 swap ，加上 * 和 & 之后真的能换动了。

外层 i 管趟数

i < n - 1 ： n 个数排 n-1 趟， i 从 0 开始。

内层 j 管比较

j < n-1-i ：末尾 i 个已归位，不再碰它们。

数组传参不用写长度

形参写 int a[] ，元素个数要另用一个 n 传进来。

02

冒泡优化与选择排序

让冒泡早点收工 · 再学一种更 " 省交换 " 的排序

swapped 标志 **选择排序** **插入排序**

哈爾濱工程大学

提前有序，就早点收工

swapped 交换标志

哈尔滨工程大学

```c
void bubble(int a[], int n) {
    int i, j, swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = 0;   // 清零
        for (j = 0; j < n-1-i; j++)
            if (a[j] > a[j+1]) {
                swap(&a[j], &a[j+1]);
                swapped = 1;
            }
        // 一趟没换 = 已经有序
        if (swapped == 0) break;
    }
```

为什么要优化

本来就有序，冒泡仍会跑满 n-1 趟。

swapped 记一笔

每趟开始清 0，交换过就置 1。

一趟没换就停

说明每对都是前小后大，已排好。

最好情况很省

只跑 1 趟、比较 n-1 次就结束。

选择排序的思想

每趟选最小 · 放到前面

初始	5	3	8	1	9	整串都没排过，先在全部 5 个数里找最小的
第 1 趟	**1**	3	8	5	9	最小是 1 ，在 a[3] ；和 a[0] 交换 → 1 到位
第 2 趟	1	**3**	8	5	9	剩下 3 8 5 9 里最小是 3 ，正好在 a[1] ，不用换
第 3 趟	1	3	**5**	8	9	剩下 8 5 9 里最小是 5 ，在 a[3] ；和 a[2] 交换
第 4 趟	1	3	5	8	9	剩下 8 9 里最小是 8 ，就在 a[3] ，不用换

关键 4 趟里真正交换只有 2 次：选择排序先 " 看完整趟 " ，再动手换一次

选择排序代码

min 记下标 · 一趟一换

哈尔滨工程大学

```
void select(int a[], int n) {
    int i, j, min;
    // 每趟给位置 i 选一个数
    for (i = 0; i < n - 1; i++) {
        min = i;   // 先假设 a[i] 最小
        // 从 i+1 扫到末尾找更小
        for (j = i + 1; j < n; j++)
            if (a[j] < a[min])
                min = j;   // 记下标
        // 最小的不在 i 位才换
        if (min != i)
            swap(&a[i], &a[min]);
```

min 记下标不记值

记住最小的在第几位，扫完再换。

j 从 i+1 起步

a[i] 和自己比没意义，从后一个扫。

外层也是 n-1 趟

最后一个位置没得选，剩下的自然就是最大的。

和冒泡的分水岭

冒泡边比边换，选择比完一趟再换一次。

冒泡 vs 选择

一张表说清差别

哈尔滨工程大学

对比项	冒泡排序	选择排序
基本思路	相邻两个比，大的往后挪	每趟选最小，放到最前面
比较次数	n(n-1)/2 次，一趟趟递减	n(n-1)/2 次，和冒泡一样
交换次数	最多 n(n-1)/2 次，比较多	最多 n-1 次，少得多
稳定性直觉	相等时不换，顺序保持不变	隔空交换，可能打乱原来次序
好记程度	双重 for ，最好写最好讲	多一个 min 下标，要绕一下

结论 两种排出来的结果完全一样（都是升序）；差别只在 " 怎么换 " 和 " 换几次 "

插入排序：像整理扑克牌

浅讲 · 只要求理解思想

摸到 5	**5**	·	·	·	·	手上第一张牌，直接拿住，不用比
摸到 3	**3**	5	·	·	·	3 比 5 小，插到 5 前面
摸到 8	3	5	**8**	·	·	8 比 5 大，直接跟在后面
摸到 1	**1**	3	5	8	·	1 比前面都小，一路比一路挪，插到最前
摸到 9	1	3	5	8	**9**	9 最大，放在末尾，整手牌排好

核心特点：手里的牌始终是有序的；新摸一张，就从后往前比，比它大的都往后挪一格，挪出空位再插进去。

本课程只要求理解思想，不要求默写代码。

三种排序怎么选

入门阶段的学习顺序

哈尔滨工程大学

三种排序排出来的结果完全一样，都是从小到大；区别只在过程：谁跟谁比、什么时候换、换了多少次。

初学阶段不必纠结谁更快，先把前两种写熟。

冒泡排序

最好写、最好记

两个 for 套起来就写完，思路一句话说得清，入门先把它写熟。

选择排序

交换次数最少

每趟最多换一次，全程不超过 n-1 次交换，想省交换就选它。

插入排序

像整理扑克牌

前面的部分始终有序，新数往前插；本节看懂思路即可。

记住这一句

冒泡和选择必须能独立写出来；插入排序看懂思路就够。

写之前先问一句：这一趟结束要确定哪个位置的数？答案清楚了，循环边界自然写对。

03

查找

在一堆数里定位目标 · 从头扫 与 对半砍

顺序查找 **返回 -1** **二分查找** **low mid high**

哈爾濱工程大学

查找是什么

查找：在一组数据里判断某个值在不在；在的话，说清楚它在第几个位置。

注意查找的结果是一个「下标」，不是那个值本身——值你已经知道了，不知道的是它藏在哪。

查什么

key

要找的那个值，

习惯起名叫 key

在哪查

数组 a

从 a[0] 一直到 a[n-1]

一个个位置看过去

查到了返回

下标 i

它在第 i 个位置，

下标从 0 开始数

没查到返回

-1

约定俗成的信号：

这里没有你要的

顺序查找

从头扫到尾

哈尔滨工程大学

```c
int search(int a[], int n, int key) {
    int i;
    for (i = 0; i < n; i++)  // 从头扫
        if (a[i] == key)     // 找到了
            return i;        // 返回下标
    return -1;   // 扫完也没有
}
// 调用示例：a 中是 5 3 8 1 9 3
int pos = search(a, 6, 8);   // pos = 2
int p2  = search(a, 6, 7);   // p2  = -1
printf("%d %d", pos, p2);    // 输出 2 -1
```

一个一个比

i 从 0 走到 n-1 ，每个位置都比一次。

找到立刻返回

return 直接结束函数，后面不用再扫。

返回的是下标

给的是位置，不是那个值本身。

最大优点

数组乱着也能查，不要求有序。

找不到就返回 -1

一个约定俗成的信号

哈尔滨工程大学

为什么偏偏是 -1 ？因为下标从 0 开始，合法下标只能是 0 到 n-1 ，永远不可能是负数。

于是 -1 成了「绝无可能的位置」，拿它当 " 没找到 " 的信号最安全。

合法下标范围

0 ~ n-1

从 0 开始数，

绝不会是负数

没找到的信号

-1

不可能出现的下标，

正好拿来当暗号

判断的写法

pos < 0

和 pos == -1 等价，

两种写法都对

千万别这样写

返回 0

0 是第一个元素的位置，

不是 " 没找到 "

二分查找：猜数字游戏

每次砍掉一半

第几次	当前范围	猜中间	得到回答	剩下的范围
第 1 次	[1, 100]	50	太大了	[1, 49] 砍掉一半
第 2 次	[1, 49]	25	太小了	[26, 49] 再砍一半
第 3 次	[26, 49]	37	太小了	[38, 49] 又砍一半
第 4 次	[38, 49]	43	太大了	[38, 42] 只剩 5 个

这就是二分

每猜一次，范围就砍掉一半。 1 到 100 猜一个数最多 7 次； 1 到 100 万最多也只要 20 次。

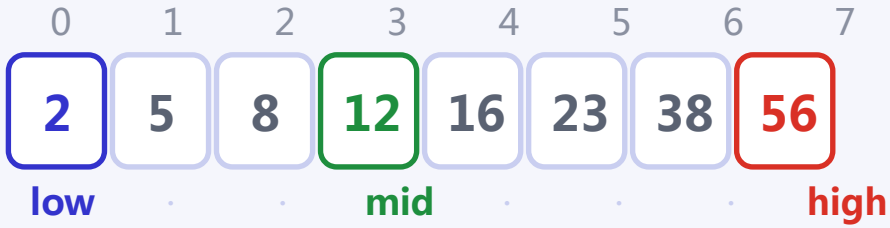
while 课那个猜数字游戏，用的就是这个思路——只不过当时是人在猜。

二分查找代码

low / mid / high

```
int bin(int a[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {        //  区间不空
        mid = (low + high) / 2;
        //  正中那个就是要找的?
        if (a[mid] == key)
            return mid;
        else if (a[mid] > key)   //  偏大
            high = mid - 1;      //  去左边
        else                     //  偏小
            low = mid + 1;       //  去右边
    }
    return -1;    //  区间空了，没有
```

三个指针各管一段



mid = (0 + 7) / 2 = 3 ； low <= high 时区间里还有一个数要查

mid 取正中

整数除法自动取整，落在正中间

区间往哪缩

偏大 high=mid-1 ，偏小 low=mid+1

二分的前提：必须有序

100 万次 vs 20 次

哈爾濱工程大學

二分查找的前提：数组必须已经排好序；乱序时「中间比大小」根本决定不了往哪边走。

记不住就记一句：先排序，后二分。

数据规模	顺序查找（最多比较）	二分查找（最多比较）
100 个	最多 100 次	最多 7 次
1 万个	最多 1 万次	最多 14 次
100 万个	最多 100 万次	最多 20 次
前提条件	乱序也能查，来者不拒	必须先排好序，否则结果不可信

例 1：冒泡排序完整程序

读入 · 排序 · 输出

题目 从键盘读入 10 个整数，用冒泡排序排成升序，再依次输出

```c
#include <stdio.h>
void swap(int *x, int *y) {
    int t = *x;
    *x = *y;
    *y = t;
}
int main(void) {
    int a[10], i, j;
    for (i = 0; i < 10; i++)
        scanf("%d", &a[i]);  // 读入
    for (i = 0; i < 9; i++)
        // 共 9 趟 = n - 1
        for (j = 0; j < 9-i; j++)
            if (a[j] > a[j+1])
                swap(&a[j], &a[j+1]);
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);
    return 0;
}
```

运行示例

输入 5 3 8 1 9 2 7 4 6 0

输出 0 1 2 3 4 5 6 7 8 9

10 个数排 9 趟，内层写 9-i

例 2：选择排序完整程序

min 记下标 一趟一换

哈尔滨工程大学

题目 读入 10 个整数，用选择排序排成升序后输出，并统计交换了几次

```c
#include <stdio.h>
int main(void) {
    int a[10], i, j, min, t;
    int cnt = 0;
    for (i = 0; i < 10; i++)
        scanf("%d", &a[i]);  // 读入
    for (i = 0; i < 9; i++) {
        min = i;   // 假设最小
        for (j = i+1; j < 10; j++)
            if (a[j] < a[min]) min = j;
        if (min != i) {  // 需要换
            t = a[i]; a[i] = a[min];
            a[min] = t;  cnt++;
        }
    }
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("交换 %d 次", cnt);
```

运行示例

输入 5 3 8 1 9 2 7 4 6 0

输出 0 1 2 3 4 5 6 7 8 9

交换次数远少于冒泡，最多 9 次

例 3：成绩排名

降序排序 + 名次表

哈尔滨工程大学

题目 5 个成绩：88 95 72 95 60 ，按从高到低排序后输出名次表

```c
#include <stdio.h>
int main(void) {
    int s[5] = {88, 95, 72, 95, 60};
    int i, j, t;
    // 冒泡：改一个符号就变降序
    for (i = 0; i < 4; i++)
        for (j = 0; j < 4-i; j++)
            if (s[j] < s[j+1]) {
                t = s[j]; s[j] = s[j+1];
                s[j+1] = t;
            }
    // 排序后依次输出名次
    for (i = 0; i < 5; i++)
        printf("%d 名 %d 分 ", i+1, s[i]);
    return 0;
}
```

输出结果（名次表）

```
1 名 95    2 名 95
3 名 88    4 名 72    5 名 60
```

排序后原下标丢失，只谈分数不谈是谁

例 4：顺序查找所有位置

不能提前 break

哈尔滨工程大学

题目 在 8 个数里找出所有等于 key 的位置并输出；一个都没有就输出「没找到」

```c
#include <stdio.h>
int main(void) {
    int a[8] = {5, 3, 8, 3, 9, 3, 1, 7};
    int key, i, cnt = 0;
    scanf("%d", &key);
    // 从头扫到尾，一处都不放过
    for (i = 0; i < 8; i++) {
        if (a[i] == key) {   // 命中
            printf("位置 %d ", i);
            cnt = cnt + 1;  // 计数
        }
    }
    // cnt 还是 0 就说明一个都没有
    if (cnt == 0) printf("没找到");
    else printf("共 %d 处", cnt);
    return 0;
}
```

运行示例

输入 3 → 位置 1 位置 3 位置 5
→ 共 3 处
输入 4 → 没找到
要找全部位置，就不能中途 break

例 5 ：二分查找与比较次数

数一数比较几次

哈爾濱工程大學

题目 在有序数组里找 key ，输出它的下标，并输出一共比较了几次

```c
#include <stdio.h>
int main(void) {
    int a[6] = {2, 5, 8, 12, 23, 56};
    int key, low = 0, high = 5, mid;
    int cnt = 0;    //  比较计数器
    scanf("%d", &key);
    //  区间不空就一直砍一半
    while (low <= high) {
        mid = (low + high) / 2;
        cnt = cnt + 1;
        if (a[mid] == key) break;
        if (a[mid] > key) high = mid - 1;
        else low = mid + 1;
    }
    //  跳出时  low<=high  才是真找到
    if (low <= high) printf(" 下标  %d", mid);
    else printf(" 没找到 ");
    printf("   比较  %d  次 ", cnt);
```

运行示例

输入 12 → 下标 3 比较 3 次
输入 100 → 没找到 比较 3 次
6 个数最多 3 次，不是 6 次

常见错误盘点

这 6 个坑，几乎人人踩过

哈尔滨工程大学

- ✗ 内层写成 j < n ， a[j+1] 直接越界 → 正确是 j < n-1-i
- ✗ 交换忘了临时变量：先赋值就把原值冲掉了 → 必须用第三个变量中转
- ✗ 把二分查找用在无序数组上 → 结果不可信，记住「先排序，后二分」
- ✗ mid 不取正中，或写成 high = mid 而不减 1 → 区间缩不动，死循环
- ✗ 趟数写死成 n 而不是 n-1 → 多跑一趟，最后一次全是白比
- ✗ 排序后还拿原来的下标去对应人 → 下标和值已经错位，编号要另存

小结

冒泡：相邻比较、大的沉底；选择：每趟选最小放前面。两个都要能独立写出来。

顺序查找从头扫到尾，找不到返回 -1 ；二分查找必须先有序，每次砍掉一半。

下一节预告

字符串处理——把一串字符当成数据来处理

哈爾濱工程大学